Practica 1 Saul de Cos Sanchez

Prompt

Moderniza este fichero a JavaScript moderno (ES2015 o posterior).

Codigo

```
/*
 * inventario.js
 * Módulo de gestión de inventario de «La Tiendecita».
 *
 * Escrito en 2012. Funciona, está en producción y nadie se atreve a tocarlo.
 * Hoy le toca a usted.
 */

const IVA = 0.21;


/* ---------- Productos ---------- */

class Producto {
  constructor(nombre, precio, stock) {
    this.nombre = nombre;
    this.precio = precio;
    this.stock = stock;
  }

  precioConIva() {
    return Math.round(this.precio * (1 + IVA) * 100) / 100;
  }

  descripcion() {
    return `${this.nombre} (${this.precioConIva()} €)`;
  }
}


/* ---------- Utilidades ---------- */

// Devuelve el valor, o el valor por defecto si no hay valor.
const valorPorDefecto = (valor, porDefecto) => valor ?? porDefecto;

// Suma todos los números que se le pasen.
const sumar = (...numeros) => numeros.reduce((total, numero) => total + numero,
0);

// Etiqueta de disponibilidad según el stock.
const clasificar = (stock) => {
  if (stock === 0) return 'agotado';
  if (stock < 5) return 'pocas unidades';
  return 'disponible';
};

// Crea una función de aviso por cada producto.
```

```
const crearAvisos = (nombres) => Array.from(
  nombres,
  (nombre) => () => `Revisa el stock de ${nombre}`
);


/* ---------- Listados ---------- */

const ordenarPorPrecio = (lista) => lista.sort((a, b) => a.precio - b.precio);

const masBarato = (lista) => {
  ordenarPorPrecio(lista);
  return lista[0];
};

const exportarCSV = (lista) => {
  const filas = ['nombre;precio;stock'];

  for (const { nombre, precio, stock } of lista) {
    filas.push(`${nombre};${precio};${stock}`);
  }

  return `${filas.join('\n')}\n`;
};


/* ---------- La tienda ---------- */

const crearTienda = (nombre) => ({
    nombre,
    productos: [],
    anadir(producto) {
      this.productos.push(producto);
      return this;
    },
    resumen() {
      return this.productos.map((producto) => `${this.nombre}:
${producto.descripcion()}`);
    },
    valorTotal() {
      const total = this.productos.reduce(
        (acumulado, producto) => acumulado + producto.precioConIva() *
producto.stock,
        0
      );
      return Math.round(total * 100) / 100;
    }
  });

// La tienda por defecto se crea al cargar el módulo.
const TIENDA_POR_DEFECTO = crearTienda('Sin nombre');


module.exports = {
  IVA,
  TIENDA_POR_DEFECTO,
  Producto,
  valorPorDefecto,
  sumar,
```

```
  clasificar,
  crearAvisos,
  ordenarPorPrecio,
  masBarato,
  exportarCSV,
  crearTienda
};
```

Salida Tests

```
node --test
✔ el módulo carga y crea la tienda por defecto (0.421375ms)
✔ Producto calcula el precio con IVA (0.066334ms)
✔ Producto genera su descripción (0.399875ms)
✔ valorPorDefecto respeta los valores que existen, aunque sean falsos (0.081917ms)
✔ valorPorDefecto sustituye null (0.094083ms)
✔ valorPorDefecto sustituye undefined (0.04475ms)
✔ sumar suma cualquier número de argumentos (0.063333ms)
✔ clasificar etiqueta el stock (0.069333ms)
✔ crearAvisos devuelve un aviso distinto para cada producto (0.130583ms)
✔ ordenarPorPrecio devuelve la lista ordenada (0.594084ms)
✔ masBarato encuentra el producto más barato (0.06275ms)
✔ la tienda añade productos y permite encadenar (0.049375ms)
✔ la tienda genera su resumen (0.0815ms)
✔ la tienda calcula su valor total (0.054333ms)
ℹ tests 14
ℹ suites 0
ℹ pass 14
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 69.826542
```

Auditoria

#	Antes → después	Caja	Por qué	Cómo lo comprobé
1	`var IVA = 0.21` → `const IVA = 0.21`	1	`IVA` no se reasigna; `const` conserva el valor y evita reasignaciones accidentales.	Busqué asignaciones a `IVA`: solo aparece la declaración.
2	Función constructora `Producto` y métodos en `prototype` → `class Producto` con `constructor` y métodos	1	Al crear productos con `new Producto(nombre, precio, stock)`, se conservan las mismas propiedades y los métodos devuelven los mismos resultados.	Los tests de precio con IVA y descripción pasan.
3	Concatenación en `descripcion` → plantilla literal	1	Con los mismos valores genera exactamente el mismo texto: `nombre (precio €)`.	El test de descripción devuelve `Taza (12.1 €)`.

#	Antes → después	Caja	Por qué	Cómo lo comprobé
4	`if (valor == null)` → `valor ?? porDefecto`	1	Ambas versiones reemplazan únicamente `null` y `undefined`, y mantienen valores falsos existentes como `0`, `false` y `''`.	Los tests cubren `null`, `undefined`, `0`, `false` y cadena vacía.
5	Bucle sobre `arguments` → `...numeros` y `reduce`	1	Los parámetros rest recogen los argumentos recibidos y `reduce` suma desde `0`, también sin argumentos.	Los tests comprueban `sumar()`, `sumar(1,2,3)` y otros casos.
6	Variable `estado` → retornos directos en `clasificar`	1	Las condiciones son las mismas: stock `0`, stock menor que `5` y resto de casos.	Los tests comprueban los tres resultados posibles.
7	Bucle con `var`, IIFE y cierre sobre índice → `Array.from` y cierre sobre `nombre` en `crearAvisos`	3	Antes cada aviso consultaba `nombres[n]` al ejecutarse; ahora guarda el nombre al crearlo. Si el array cambia después, el aviso ya no refleja el nuevo valor.	Crear un aviso, cambiar `nombres[0]` y llamar al aviso: el código moderno conserva el nombre inicial. Los tests no modifican el array.
8	Comparador con función normal → función flecha en `ordenarPorPrecio`	1	El comparador sigue restando `a.precio - b.precio`, por lo que mantiene el mismo criterio y ordenación.	El test de ordenación pasa.
9	Función normal → función flecha en `masBarato`	1	Sigue ordenando la lista mediante `ordenarPorPrecio` y devolviendo el primer elemento.	El test de producto más barato pasa.
10	Construcción manual del CSV con índices y concatenación → `for...of`, array de filas y plantillas literales	1	Para una lista de productos, conserva cabecera, orden, separadores `;` y salto final de línea.	Comparé la salida de una lista de prueba con `nombre;precio;stock\nTaza;10;3\n`, carácter a carácter.
11	`function crearTienda` y propiedades con `function` → función flecha, propiedad abreviada y métodos concisos	1	La tienda conserva `nombre`, `productos`, el encadenamiento de `anadir`, el resumen y el cálculo del valor total. La flecha de `resumen` sustituye correctamente a la variable auxiliar `self`.	Los tests de creación, encadenamiento, resumen y valor total pasan.
12	`var TIENDA_POR_DEFECTO = ...` antes de las funciones → `const`	1	Sigue creándose una única tienda llamada `Sin nombre` al cargar el módulo; solo	El test de carga del módulo comprueba la tienda por defecto.

#	Antes → después	Caja	Por qué	Cómo lo comprobé
	`TIENDA_POR_DEFECTO = ...` después de `crearTienda`		cambia su posición para poder usar `const`.	
13	Exportaciones explícitas, como `IVA: IVA` → propiedades abreviadas, como `IVA`	1	Las claves y valores de `module.exports` son los mismos, así que el código que usa `require('./inventario')` mantiene la misma interfaz.	Los 14 tests importan el módulo y pasan.

Version final

```
/*
 * inventario.js
 * Módulo de gestión de inventario de «La Tiendecita».
 *
 * Escrito en 2012. Funciona, está en producción y nadie se atreve a tocarlo.
 * Hoy le toca a usted.
 */

const IVA = 0.21;


/* ---------- Productos ---------- */

class Producto {
  constructor(nombre, precio, stock) {
    this.nombre = nombre;
    this.precio = precio;
    this.stock = stock;
  }

  precioConIva() {
    return Math.round(this.precio * (1 + IVA) * 100) / 100;
  }

  descripcion() {
    return `${this.nombre} (${this.precioConIva()} €)`;
  }
}


/* ---------- Utilidades ---------- */

// Devuelve el valor, o el valor por defecto si no hay valor.
const valorPorDefecto = (valor, porDefecto) => valor ?? porDefecto;

// Suma todos los números que se le pasen.
const sumar = (...numeros) => numeros.reduce((total, numero) => total + numero,
0);

// Etiqueta de disponibilidad según el stock.
const clasificar = (stock) => {
```

```
  if (stock === 0) return 'agotado';
  if (stock < 5) return 'pocas unidades';
  return 'disponible';
};

// Crea una función de aviso por cada producto.
const crearAvisos = (nombres) => Array.from(
  nombres,
  (nombre) => () => `Revisa el stock de ${nombre}`
);


/* ---------- Listados ---------- */

const ordenarPorPrecio = (lista) => lista.sort((a, b) => a.precio - b.precio);

const masBarato = (lista) => {
  ordenarPorPrecio(lista);
  return lista[0];
};

const exportarCSV = (lista) => {
  const filas = ['nombre;precio;stock'];

  for (const { nombre, precio, stock } of lista) {
    filas.push(`${nombre};${precio};${stock}`);
  }

  return `${filas.join('\n')}\n`;
};


/* ---------- La tienda ---------- */

const crearTienda = (nombre) => ({
    nombre,
    productos: [],
    anadir(producto) {
      this.productos.push(producto);
      return this;
    },
    resumen() {
      return this.productos.map((producto) => `${this.nombre}:
${producto.descripcion()}`);
    },
    valorTotal() {
      const total = this.productos.reduce(
        (acumulado, producto) => acumulado + producto.precioConIva() *
producto.stock,
        0
      );
      return Math.round(total * 100) / 100;
    }
  });

// La tienda por defecto se crea al cargar el módulo.
const TIENDA_POR_DEFECTO = crearTienda('Sin nombre');
```

```
module.exports = {
  IVA,
  TIENDA_POR_DEFECTO,
  Producto,
  valorPorDefecto,
  sumar,
  clasificar,
  crearAvisos,
  ordenarPorPrecio,
  masBarato,
  exportarCSV,
  crearTienda
};
```

Declaraciones

Autoría

Declaro que he utilizado Codex como apoyo para proponer la implementación. He revisado el código, comprobado su funcionamiento y entiendo los cambios incluidos en esta entrega.

Uso IA

IA Utilizada	Uso en esta entrega
Codex (Terra 5.6)	Prompt indicado en el enunciado y pequenas aclaraciones del codigo devuelto y creacion de tabla base del apartado de auditoria